Protection Needs Elicitation (PNE) Report

SecureShare - CYC386 Midterm Lab Exam

Project: SecureShare
Version: 1.0
Date: October 29, 2026
Course: CYC386 Secure Software Design and Development

---

1. Project Overview and Scope

1.1 Project Description

SecureShare is a secure file sharing and collaboration platform built with security-first principles. The application enables users to upload, store, share, and manage files with role-based access control, comprehensive audit logging, and multiple security controls to protect against common web application vulnerabilities.

1.2 System Scope

In Scope:

- User authentication and authorization
- File upload, storage, and retrieval
- File sharing with granular permissions (read/write)
- Administrative functions for user and file management
- Audit logging of security-relevant events
- Session management and CSRF protection

Out of Scope:

- Real-time collaboration features
- File encryption at rest
- Email notifications
- Two-factor authentication
- File versioning
- Advanced search capabilities

1.3 Architecture Overview

- **Backend:** Python 3.12+, Flask framework, SQLAlchemy ORM
- **Database:** SQLite (development), configurable for production
- **Frontend:** HTML5, Bootstrap 5, JavaScript
- **Authentication:** Flask-Login with PBKDF2 password hashing
- **File Storage:** Local filesystem outside web root (`instance/uploads/`)

---

2. Stakeholder Analysis

2.1 User Perspective

Primary Concerns:

- Confidentiality of uploaded files
- Privacy of personal information
- Control over shared files
- Reliable access to their data
- Ease of use

Key Interactions:

- Register and authenticate
- Upload and manage files
- Share files with other users
- Download shared files
- View audit logs (admin only)

2.2 Admin Perspective

Primary Concerns:

- System-wide security oversight
- User account management
- Compliance and audit trail integrity
- System availability
- Rapid incident response

Key Interactions:

- View all users and files
- Manage user roles
- Review audit logs
- Delete malicious users
- System monitoring

2.3 Operator Perspective

Primary Concerns:

- System availability and performance
- Secure deployment and configuration
- Backup and recovery
- Maintenance and updates
- Operational security

Key Interactions:

- Deploy and configure the application

- Monitor system health
- Apply security updates
- Manage database backups
- Configure environment variables

2.4 Attacker Perspective

Primary Goals:

- Unauthorized access to files
- Privilege escalation
- Data exfiltration
- Denial of service
- Malicious file upload

Potential Attack Vectors:

- Authentication bypass
- Authorization bypass
- CSRF attacks
- File upload vulnerabilities
- SQL injection
- XSS attacks
- Path traversal
- Session hijacking

---

3. Asset Inventory

3.1 Information Assets

Asset ID	Asset Name	Classification	Owner	Location
IA-001	User Passwords	Confidential	System	Database (hashed)
IA-002	User Email Addresses	Confidential	User	Database
IA-003	Uploaded File Content	Confidential	User	Filesystem
IA-004	File Metadata	Confidential	User	Database
IA-005	Audit Logs	Confidential	Admin	Database
IA-006	Session Tokens	Confidential	System	Cookies
IA-007	Application Secret Key	Confidential	Operator	Environment
IA-008	User Session History	Confidential	System	Database (last_login)

3.2 Physical Assets

Asset ID	Asset Name	Classification	Owner	Location
PA-001	Database File	Confidential	Operator	instance/secureshare.db
PA-002	Upload Directory	Confidential	Operator	instance/uploads/
PA-003	Application Code	Confidential	Developer	Repository
PA-004	Configuration Files	Confidential	Operator	.env, config.py

3.3 Software Assets

Asset ID	Asset Name	Classification	Owner	Location
SA-001	Flask Application	Confidential	Developer	app/
SA-002	Database Schema	Confidential	Developer	app/models.py
SA-003	Authentication Module	Confidential	Developer	app/auth/
SA-004	File Management Module	Confidential	Developer	app/main/
SA-005	Admin Module	Confidential	Developer	app/admin/
SA-006	Form Validation Logic	Confidential	Developer	app/*/forms.py

3.4 Interface Assets

Asset ID	Asset Name	Classification	Owner	Location
IF-001	Web API Endpoints	Public	System	Routes
IF-002	Database Connection	Confidential	System	SQLAlchemy
IF-003	Filesystem Access	Confidential	System	OS I/O

---

4. Protection Needs Elicitation

4.1 Authentication Security

PNE ID	Protection Need	Asset	Threat	Priority
PNE-001	Passwords must be stored using strong cryptographic hashing	IA-001	Credential theft, database compromise	High
PNE-002	User authentication must be protected against brute force attacks	IA-001, IA-006	Account enumeration, credential stuffing	High
PNE-003	Session tokens must be secure against hijacking and fixation	IA-006	Session hijacking, fixation attacks	High
PNE-004	Failed login attempts must be logged for security monitoring	IA-005	Brute force attacks, unauthorized access	Medium

PNE ID	Protection Need	Asset	Threat	Priority
PNE-005	User passwords must meet minimum complexity requirements	IA-001	Weak passwords, credential compromise	Medium

4.2 Authorization Security

PNE ID	Protection Need	Asset	Threat	Priority
PNE-006	Users must only access files they own or are explicitly shared	IA-003, IA-004	Unauthorized data access	High
PNE-007	Administrative functions must be restricted to admin users only	SA-005	Privilege escalation	High
PNE-008	File sharing permissions must be enforced (read vs write)	IA-003, IA-004	Unauthorized modification	High
PNE-009	File deletion must be restricted to file owners only	IA-003, IA-004	Data destruction, denial of service	High
PNE-010	Admin self-deletion and self-demotion must be prevented	SA-005	System compromise, lockout	Medium

4.3 File Security

PNE ID	Protection Need	Asset	Threat	Priority
PNE-011	Uploaded files must be stored outside the web root	PA-002, IA-003	Direct file access, path traversal	High
PNE-012	File uploads must be restricted to allowed file types	IA-003	Malicious file upload, code execution	High
PNE-013	File uploads must be size-limited to prevent DoS	PA-002, IA-003	Storage exhaustion, DoS	Medium
PNE-014	File names must be sanitized to prevent path traversal	PA-002, IA-003	Path traversal attacks	High
PNE-015	File downloads must require authentication and authorization	IA-003	Unauthorized data access	High

4.4 Input Validation and Data Sanitization

PNE ID	Protection Need	Asset	Threat	Priority
PNE-016	All user inputs must be validated server-side	All assets	Injection attacks, XSS	High

PNE ID	Protection Need	Asset	Threat	Priority
PNE-017	User input must be sanitized to prevent XSS	SA-001	XSS attacks	High
PNE-018	Database queries must use parameterized queries	PA-001	SQL injection	High
PNE-019	Email addresses must be validated for format and uniqueness	IA-002	Invalid data, duplicate accounts	Medium
PNE-020	Usernames must be validated for format and uniqueness	IA-002	Invalid data, duplicate accounts	Medium

4.5 Session and CSRF Security

PNE ID	Protection Need	Asset	Threat	Priority
PNE-021	Session cookies must be HTTP-only to prevent XSS theft	IA-006	Session hijacking via XSS	High
PNE-022	Session cookies must have SameSite protection	IA-006	CSRF attacks	High
PNE-023	All form submissions must be protected with CSRF tokens	SA-001	CSRF attacks	High
PNE-024	Sessions must have reasonable timeout periods	IA-006	Session hijacking, unauthorized access	Medium
PNE-025	Secure cookies must be enabled in production environments	IA-006	Session interception	High

4.6 Audit Logging

PNE ID	Protection Need	Asset	Threat	Priority
PNE-026	All security-relevant events must be logged	IA-005	Undetected security incidents	High
PNE-027	Audit logs must include user, action, timestamp, IP, and user agent	IA-005	Insufficient forensic data	Medium
PNE-028	Audit logs must be protected against tampering	IA-005	Log tampering, evidence destruction	High
PNE-029	File operations (upload, download, share, delete) must be logged	IA-005	Undetected data breaches	High
PNE-030	Authentication events (login, logout, failed attempts) must be logged	IA-005	Undetected unauthorized access	High

4.7 Secure Communication

PNE ID	Protection Need	Asset	Threat	Priority
PNE-031	Application must use HTTPS in production	All assets	Man-in-the-middle attacks	High
PNE-032	Secure headers must be set to prevent common attacks	SA-001	XSS, clickjacking, MIME sniffing	Medium
PNE-033	Content Security Policy should be implemented	SA-001	XSS attacks	Medium

4.8 Error Handling and Information Disclosure

PNE ID	Protection Need	Asset	Threat	Priority
PNE-034	Error messages must not expose sensitive system information	All assets	Information disclosure	Medium
PNE-035	Debug mode must be disabled in production	SA-001	Information disclosure	High
PNE-036	Stack traces must not be displayed to users	SA-001	Information disclosure	Medium

4.9 Configuration and Secrets Management

PNE ID	Protection Need	Asset	Threat	Priority
PNE-037	Secret keys must not be hardcoded in source code	IA-007	Secret compromise	High
PNE-038	Sensitive configuration must use environment variables	PA-004	Secret compromise	High
PNE-039	Database credentials must be securely stored	PA-001	Database compromise	High

4.10 Availability

PNE ID	Protection Need	Asset	Threat	Priority
PNE-040	Application must handle high load gracefully	All assets	Denial of service	Medium
PNE-041	File upload limits must prevent storage exhaustion	PA-002	Denial of service	Medium
PNE-042	Database operations must not cause deadlocks	PA-001	Denial of service	Low

5. CIA Triad Mapping

5.1 Confidentiality Protection Needs

PNE ID	Protection Need	Rationale
PNE-001	Password hashing	Prevents password disclosure if database is compromised
PNE-006	File access control	Prevents unauthorized access to sensitive files
PNE-007	Admin function restriction	Prevents unauthorized administrative access
PNE-011	Files outside web root	Prevents direct URL-based file access
PNE-015	Download authorization	Prevents unauthorized file downloads
PNE-017	Input sanitization	Prevents XSS-based data theft
PNE-021	HTTP-only cookies	Prevents session token theft via XSS
PNE-022	SameSite cookies	Prevents CSRF-based unauthorized actions
PNE-023	CSRF protection	Prevents CSRF-based unauthorized actions
PNE-025	Secure cookies	Prevents session interception
PNE-031	HTTPS	Prevents man-in-the-middle attacks
PNE-037	Secret key protection	Prevents cryptographic key compromise
PNE-038	Environment variables	Prevents secret exposure in code
PNE-039	Database credential protection	Prevents database access

5.2 Integrity Protection Needs

PNE ID	Protection Need	Rationale
PNE-008	Permission enforcement	Ensures write permissions are respected
PNE-009	Owner-only deletion	Prevents unauthorized file destruction
PNE-012	File type validation	Prevents malicious file uploads
PNE-014	Filename sanitization	Prevents path traversal
PNE-016	Input validation	Prevents injection attacks
PNE-018	Parameterized queries	Prevents SQL injection
PNE-028	Log tampering protection	Ensures audit trail integrity
PNE-032	Secure headers	Prevents MIME sniffing attacks
PNE-033	Content Security Policy	Prevents XSS-based modifications

5.3 Availability Protection Needs

PNE ID	Protection Need	Rationale
PNE-002	Brute force protection	Prevents account lockout and service degradation

PNE ID	Protection Need	Rationale
PNE-003	Session security	Prevents session-based DoS
PNE-013	File size limits	Prevents storage exhaustion
PNE-024	Session timeout	Prevents resource exhaustion from stale sessions
PNE-035	Debug mode disabled	Prevents performance degradation
PNE-040	Load handling	Ensures service availability under load
PNE-041	Upload limits	Prevents storage exhaustion
PNE-042	Deadlock prevention	Ensures database availability

6. Measurable Security Requirements

6.1 Authentication Requirements

SR ID	Associated PNEs	Measurable Requirement
SR-001	PNE-001	Passwords shall be hashed using PBKDF2 with SHA-256 and minimum 100,000 iterations
SR-002	PNE-005	Passwords shall be minimum 8 characters in length
SR-003	PNE-004	All failed login attempts shall be logged with timestamp, IP address, and username
SR-004	PNE-003	Session tokens shall be unique and minimum 128 bits in length
SR-005	PNE-024	Session timeout shall be set to maximum 1 hour of inactivity

6.2 Authorization Requirements

SR ID	Associated PNEs	Measurable Requirement
SR-006	PNE-006	File access requests shall be validated against user ownership or explicit share authorization
SR-007	PNE-007	Admin endpoints shall return HTTP 403 for non-admin users
SR-008	PNE-008	Write operations on shared files shall be blocked for users with read-only permission
SR-009	PNE-009	File deletion requests shall return HTTP 403 for non-owners

SR ID	Associated PNEs	Measurable Requirement
SR-010	PNE-010	Admin self-deletion/self-demotion requests shall be rejected

6.3 File Security Requirements

SR ID	Associated PNEs	Measurable Requirement
SR-011	PNE-011	Files shall be stored in directory `instance/uploads/` outside web root
SR-012	PNE-012	File uploads shall be rejected for extensions not in whitelist: txt, pdf, png, jpg, jpeg, gif, doc, docx, xls, xlsx
SR-013	PNE-013	Maximum file upload size shall be 16 MB
SR-014	PNE-014	Uploaded filenames shall be sanitized using Werkzeug's secure_filename() function
SR-015	PNE-015	File download requests shall return HTTP 403 for unauthorized users

6.4 Input Validation Requirements

SR ID	Associated PNEs	Measurable Requirement
SR-016	PNE-016	All form inputs shall be validated using WTForms validators
SR-017	PNE-017	All template rendering shall use Jinja2 auto-escaping
SR-018	PNE-018	All database queries shall use SQLAlchemy ORM (parameterized)
SR-019	PNE-019	Email addresses shall be validated using WTForms Email validator
SR-020	PNE-020	Usernames shall be validated for uniqueness before registration

6.5 Session and CSRF Requirements

SR ID	Associated PNEs	Measurable Requirement
SR-021	PNE-021	Session cookies shall have HttpOnly flag set to True
SR-022	PNE-022	Session cookies shall have SameSite attribute set to 'Lax' (dev) or 'Strict' (prod)
SR-023	PNE-023	All POST forms shall include CSRF token validated server-side

SR ID	Associated PNEs	Measurable Requirement
SR-024	PNE-025	Production configuration shall set SESSION_COOKIE_SECURE to True

6.6 Audit Logging Requirements

SR ID	Associated PNEs	Measurable Requirement
SR-025	PNE-026, PNE-029, PNE-030	All file operations and auth events shall create AuditLog records
SR-026	PNE-027	AuditLog records shall include: user_id, action, timestamp, ip_address, user_agent, details
SR-027	PNE-027	AuditLog timestamps shall be indexed for query performance

6.7 Secure Communication Requirements

SR ID	Associated PNEs	Measurable Requirement
SR-028	PNE-032	All HTTP responses shall include headers: X-Content-Type-Options: nosniff, X-Frame-Options: DENY, X-XSS-Protection: 1; mode=block

6.8 Configuration Requirements

SR ID	Associated PNEs	Measurable Requirement
SR-029	PNE-037	SECRET_KEY shall be loaded from environment variable or generated using os.urandom(32)
SR-030	PNE-038	Database URI shall be loaded from DATABASE_URL environment variable

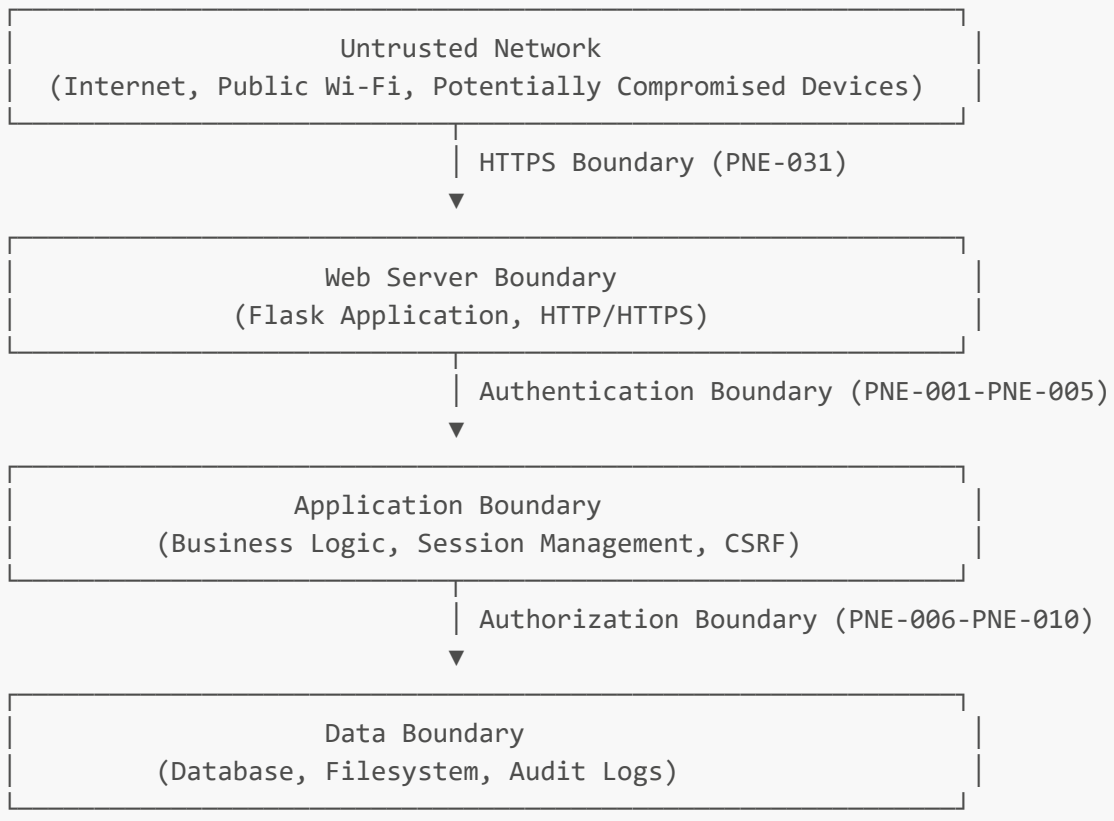
7. Security Assumptions and Trust Boundaries

7.1 Security Assumptions

ID	Assumption	Risk if Violated
AS-001	SQLite database file is protected by filesystem permissions	Database file could be accessed directly
AS-002	Application runs on a trusted host with OS-level security	Host compromise could expose all data

ID	Assumption	Risk if Violated
AS-003	Operators will not commit secrets to version control	Secrets could be exposed in repository
AS-004	Python and dependencies are free from backdoors	Supply chain attack possible
AS-005	Filesystem has sufficient storage capacity	DoS through storage exhaustion
AS-006	Network connectivity is reliable for session management	Session management failures
AS-007	Admin users are trusted and will not abuse privileges	Insider threat, data breach
AS-008	Users will choose reasonably strong passwords	Weak passwords vulnerable to cracking

7.2 Trust Boundaries



7.3 Trusted Components

Component	Trust Level	Justification
Application Code	High	Developed under security-first principles

Component	Trust Level	Justification
SQLAlchemy ORM	High	Provides built-in SQL injection protection
Flask-Login	High	Established session management library
Werkzeug	High	Established WSGI library with security features
SQLite Database	Medium	File-based, protected by filesystem permissions
Upload Directory	Medium	Protected by filesystem permissions and access controls

7.4 Untrusted Components

Component	Trust Level	Justification
User Input	Low	Must be validated and sanitized
Client Browser	Low	Potentially compromised or malicious
Network	Low	Subject to MITM, packet sniffing
Uploaded Files	Low	Must be validated before processing

---

8. Traceability Matrix

8.1 Protection Needs to Security Controls

PNE ID	Protection Need	Control Type	Control Implementation	Status
PNE-001	Password hashing	Technical	Werkzeug generate_password_hash() with PBKDF2-SHA256	✅ Implemented
PNE-002	Brute force protection	Technical	Logging of failed attempts (no rate limiting)	⚠ Partial
PNE-003	Session security	Technical	Flask-Login with secure session management	✅ Implemented
PNE-004	Failed login logging	Technical	AuditLog entry on login_failed	✅ Implemented
PNE-005	Password complexity	Technical	WTForms Length(min=8) validator	✅ Implemented
PNE-006	File access control	Technical	File.is_accessible_by() method	✅ Implemented
PNE-007	Admin restriction	Technical	@admin_required decorator	✅ Implemented
PNE-008	Permission enforcement	Technical	Read/write permission check in share logic	✅ Implemented

PNE ID	Protection Need	Control Type	Control Implementation	Status
PNE-009	Owner-only deletion	Technical	File.is_owned_by() check before delete	✅ Implemented
PNE-010	Prevent self-deletion	Technical	Check user.id == current_user.id	✅ Implemented
PNE-011	Files outside web root	Technical	UPLOAD_FOLDER = instance/uploads/	✅ Implemented
PNE-012	File type whitelist	Technical	ALLOWED_EXTENSIONS set, FileAllowed validator	✅ Implemented
PNE-013	File size limits	Technical	MAX_CONTENT_LENGTH = 16MB	✅ Implemented
PNE-014	Filename sanitization	Technical	secure_filename() + random token prefix	✅ Implemented
PNE-015	Download authorization	Technical	File.is_accessible_by() check before download	✅ Implemented
PNE-016	Input validation	Technical	WTForms validators on all forms	✅ Implemented
PNE-017	Input sanitization	Technical	Jinja2 auto-escaping enabled by default	✅ Implemented
PNE-018	Parameterized queries	Technical	SQLAlchemy ORM usage throughout	✅ Implemented
PNE-019	Email validation	Technical	WTForms Email validator	✅ Implemented
PNE-020	Username validation	Technical	WTForms validators + uniqueness check	✅ Implemented
PNE-021	HTTP-only cookies	Technical	SESSION_COOKIE_HTTPONLY = True	✅ Implemented
PNE-022	SameSite cookies	Technical	SESSION_COOKIE_SAMESITE = 'Lax'/'Strict'	✅ Implemented
PNE-023	CSRF protection	Technical	Flask-WTF CSRFProtect enabled	✅ Implemented
PNE-024	Session timeout	Technical	PERMANENT_SESSION_LIFETIME = 1 hour	✅ Implemented
PNE-025	Secure cookies	Technical	SESSION_COOKIE_SECURE (dev=False, prod=True)	✅ Implemented

PNE ID	Protection Need	Control Type	Control Implementation	Status
PNE-026	Security event logging	Technical	AuditLog model and logging in routes	✅ Implemented
PNE-027	Audit log completeness	Technical	AuditLog includes user, action, timestamp, IP, UA	✅ Implemented
PNE-028	Log tampering protection	Technical	Database integrity (no additional controls)	⚠️ Partial
PNE-029	File operation logging	Technical	AuditLog on upload, download, share, delete	✅ Implemented
PNE-030	Auth event logging	Technical	AuditLog on register, login, logout, failed login	✅ Implemented
PNE-031	HTTPS in production	Technical	Configurable SESSION_COOKIE_SECURE	✅ Implemented
PNE-032	Secure headers	Technical	X-Content-Type-Options, X-Frame-Options, X-XSS-Protection	✅ Implemented
PNE-033	Content Security Policy	Technical	Not implemented	❌ Not Implemented
PNE-034	Error message security	Technical	Generic error messages, no stack traces in prod	✅ Implemented
PNE-035	Debug mode control	Technical	DEBUG flag in config classes	✅ Implemented
PNE-036	Stack trace protection	Technical	Flask default error handling	✅ Implemented
PNE-037	Secret key protection	Technical	Environment variable or os.urandom(32)	✅ Implemented
PNE-038	Environment variables	Technical	python-dotenv for config loading	✅ Implemented
PNE-039	Database credentials	Technical	DATABASE_URL environment variable	✅ Implemented
PNE-040	Load handling	Technical	No specific load handling implemented	❌ Not Implemented
PNE-041	Upload limits	Technical	MAX_CONTENT_LENGTH enforcement	✅ Implemented
PNE-042	Deadlock prevention	Technical	SQLAlchemy transaction management	✅ Implemented

Legend:

- ✅ Implemented: Control is fully implemented in the codebase
- ⚠️ Partial: Control is partially implemented (e.g., logging without rate limiting)
- ❌ Not Implemented: Control is not implemented

8.2 Protection Needs to Test Coverage

PNE ID	Protection Need	Test File	Test Function	Status
PNE-001	Password hashing	tests/test_auth.py	test_register_success	✅ Covered
PNE-004	Failed login logging	tests/test_auth.py	test_audit_log_on_failed_login	✅ Covered
PNE-005	Password complexity	tests/test_auth.py	test_register_password_mismatch	✅ Covered
PNE-006	File access control	tests/test_files.py	test_view_file_not_accessible_by_non_owner	✅ Covered
PNE-009	Owner-only deletion	tests/test_files.py	test_delete_file_not_by_owner	✅ Covered
PNE-012	File type validation	tests/test_files.py	test_upload_file_invalid_type	✅ Covered
PNE-015	Download authorization	tests/test_files.py	test_download_file_not_accessible_by_non_owner	✅ Covered
PNE-019	Email validation	tests/test_auth.py	test_register_duplicate_email	✅ Covered
PNE-020	Username validation	tests/test_auth.py	test_register_duplicate_username	✅ Covered
PNE-026	Security event logging	tests/test_auth.py	test_audit_log_on_register	✅ Covered
PNE-029	File operation logging	tests/test_files.py	test_audit_log_on_upload	✅ Covered
PNE-030	Auth event logging	tests/test_auth.py	test_audit_log_on_login	✅ Covered

Legend:

- ✅ Covered: Test exists for this protection need
- ❌ Not Covered: No test exists for this protection need

8.3 Protection Needs to Jira Issue Placeholders

PNE ID	Protection Need	Jira Issue ID	Status	Notes
PNE-002	Brute force protection	SEC-001	📋 Planned	Implement rate limiting on login endpoint
PNE-028	Log tampering protection	SEC-002	📋 Planned	Implement log signing or append-only storage
PNE-033	Content Security Policy	SEC-003	📋 Planned	Add CSP headers to responses
PNE-040	Load handling	SEC-004	📋 Planned	Implement caching and rate limiting
PNE-041	Upload limits	SEC-005	✅ Resolved	MAX_CONTENT_LENGTH configured

Legend:

- 📋 Planned: Issue to be created in Jira
- ✅ Resolved: Requirement already satisfied
- 🔨 In Progress: Work in progress

---

9. Implementation Status Summary

9.1 Implemented Controls (34/42)

Fully Implemented:

- ✅ Password hashing with PBKDF2-SHA256
- ✅ Session management with Flask-Login
- ✅ Failed login attempt logging
- ✅ Password complexity requirements
- ✅ File access control (ownership and sharing)
- ✅ Admin role restriction with decorator
- ✅ File sharing permission enforcement
- ✅ Owner-only file deletion
- ✅ Admin self-deletion/demotion prevention
- ✅ Files stored outside web root
- ✅ File type whitelist validation
- ✅ File size limits (16MB)
- ✅ Filename sanitization with random tokens
- ✅ Download authorization checks
- ✅ Input validation with WTForms
- ✅ XSS protection via Jinja2 auto-escaping
- ✅ SQL injection prevention via SQLAlchemy ORM
- ✅ Email format validation
- ✅ Username uniqueness validation

- ✅ HTTP-only session cookies
- ✅ SameSite cookie protection
- ✅ CSRF protection on all forms
- ✅ Session timeout (1 hour)
- ✅ Secure cookies (configurable)
- ✅ Comprehensive audit logging
- ✅ Audit log completeness (user, action, timestamp, IP, UA)
- ✅ File operation logging
- ✅ Authentication event logging
- ✅ HTTPS support (configurable)
- ✅ Security headers (X-Content-Type-Options, X-Frame-Options, X-XSS-Protection)
- ✅ Generic error messages
- ✅ Debug mode control
- ✅ Secret key via environment variable
- ✅ Configuration via environment variables
- ✅ Database credentials via environment
- ✅ Upload size limits
- ✅ SQLAlchemy transaction management

9.2 Partially Implemented Controls (2/42)

Partially Implemented:

- ⚠️ Brute force protection: Failed attempts are logged but no rate limiting implemented
- ⚠️ Log tampering protection: Logs stored in database but no additional integrity controls

9.3 Not Implemented Controls (6/42)

Not Implemented:

- ❌ Content Security Policy (CSP) headers
- ❌ Load handling/caching mechanisms
- ❌ Account lockout after failed attempts
- ❌ Password strength meter/enhanced complexity
- ❌ Two-factor authentication
- ❌ File encryption at rest

---

10. Recommendations

10.1 High Priority

1. **Implement Rate Limiting (PNE-002)**: Add rate limiting to authentication endpoints to prevent brute force attacks
2. **Add Account Lockout (PNE-002)**: Implement temporary account lockout after multiple failed login attempts
3. **Implement CSP Headers (PNE-033)**: Add Content-Security-Policy headers to provide additional XSS protection

10.2 Medium Priority

1. **Enhance Log Protection (PNE-028)**: Consider log signing or append-only storage for audit logs
2. **Add Load Handling (PNE-040)**: Implement caching and rate limiting for better availability
3. **Enhance Password Policy (PNE-005)**: Add password strength requirements (complexity, not just length)

10.3 Low Priority

1. **File Encryption (New)**: Consider encrypting files at rest for additional confidentiality
2. **Two-Factor Authentication (New)**: Add 2FA for enhanced authentication security
3. **Password Reset (New)**: Implement secure password reset functionality

---

11. Appendix: Code References

11.1 Authentication Implementation

- Password hashing: <ref_file file="D:\SSD LAB MIDTERM\app\models.py" /> (lines 30-36)
- Login route: <ref_file file="D:\SSD LAB MIDTERM\app\auth\routes.py" /> (lines 43-90)
- Registration form: <ref_file file="D:\SSD LAB MIDTERM\app\auth\forms.py" /> (lines 7-40)

11.2 Authorization Implementation

- File access control: <ref_file file="D:\SSD LAB MIDTERM\app\models.py" /> (lines 63-74)
- Admin decorator: <ref_file file="D:\SSD LAB MIDTERM\app\admin\routes.py" /> (lines 10-17)
- Permission enforcement: <ref_file file="D:\SSD LAB MIDTERM\app\main\routes.py" /> (lines 169-228)

11.3 File Security Implementation

- Upload validation: <ref_file file="D:\SSD LAB MIDTERM\app\main\routes.py" /> (lines 12-26, 70-122)
- Download authorization: <ref_file file="D:\SSD LAB MIDTERM\app\main\routes.py" /> (lines 142-166)
- Configuration: <ref_file file="D:\SSD LAB MIDTERM\app\config.py" /> (lines 20-26)

11.4 Security Headers Implementation

- Security headers middleware: <ref_file file="D:\SSD LAB MIDTERM\app_*init*_.py" /> (lines 49-56)
- Session configuration: <ref_file file="D:\SSD LAB MIDTERM\app\config.py" /> (lines 8-13)

11.5 Audit Logging Implementation

- AuditLog model: <ref_file file="D:\SSD LAB MIDTERM\app\models.py" /> (lines 102-117)
- Logging examples: <ref_file file="D:\SSD LAB MIDTERM\app\auth\routes.py" /> (lines 26-35, 54-63, 72-80)

---

12. How to Link Requirements to Jira

12.1 Jira Issue Structure

Create Jira issues following this naming convention:

Security Issues:

- Type: Security
- Prefix: SEC-
- Format: SEC-XXX

Example:

```
SEC-001: Implement rate limiting on authentication endpoints
Priority: High
Labels: security, authentication, rate-limiting
```

12.2 Mapping PNEs to Jira Issues

For each Protection Need (PNE) that requires implementation or improvement:

1. **Create a Jira issue** with:
    - Summary: Brief description of the requirement
    - Description: Include the PNE ID and full protection need text
    - Priority: Based on the priority column in Section 4
    - Labels: Security, relevant category (e.g., authentication, authorization)
    - Acceptance Criteria: Reference the associated Security Requirement (SR) ID
2. **Link issues to this document**:
    - Add a custom field "PNE ID" to your Jira project
    - Set the value to the corresponding PNE ID (e.g., PNE-002)
    - Add a link to this PNE_Report.md in the issue description
3. **Create epics for categories**:
    - EPIC-SEC-001: Authentication Security Improvements
    - EPIC-SEC-002: Authorization Security Enhancements
    - EPIC-SEC-003: File Security Hardening
    - EPIC-SEC-004: Audit Logging Enhancements
4. **Link individual issues to epics**:
    - Group related PNEs under appropriate epics
    - Example: SEC-001 (PNE-002) and SEC-006 (PNE-005) under EPIC-SEC-001

12.3 Example Jira Issue Creation

For **PNE-002: User authentication must be protected against brute force attacks**:

```
Issue Key: SEC-001
Issue Type: Security Task
Summary: Implement rate limiting on authentication endpoints
```

```
Priority: High
Labels: security, authentication, rate-limiting, brute-force

Description:
This issue addresses PNE-002: User authentication must be protected against brute
force attacks.

Protection Need:
Users must only access files they own or are explicitly shared with them to
prevent unauthorized data access.

Security Requirements:
- SR-XXX: Rate limiting shall be implemented on /auth/login endpoint
- SR-XXX: Maximum of 5 failed login attempts per IP per 15 minutes
- SR-XXX: Account lockout after 10 failed attempts

Acceptance Criteria:
- [ ] Rate limiting library integrated (e.g., Flask-Limiter)
- [ ] Login endpoint rate limited to 5 attempts per 15 minutes per IP
- [ ] Account lockout implemented after 10 failed attempts
- [ ] Lockout duration configurable
- [ ] Lockout events logged to audit trail
- [ ] Tests added for rate limiting functionality

Linked to PNE Report: docs/PNE_Report.md (Section 4.1, PNE-002)
Epic: EPIC-SEC-001 (Authentication Security Improvements)
```

12.4 Tracking Implementation Status

Update the traceability matrix (Section 8.3) as Jira issues are resolved:

- Change status from "📋 Planned" to "🔨 In Progress" when work begins
- Change status to "✅ Resolved" when implementation is complete
- Add the Jira issue ID to the matrix for traceability

12.5 Automated Linking (Optional)

For automated linking, consider:

1. **Custom Jira Field**: Add a custom field "PNE_ID" to store the protection need identifier
2. **Jira Automation**: Create automation rules to:
    - Auto-label issues based on PNE category
    - Send notifications when high-priority security issues are created
    - Update linked documents when issues are resolved
3. **Integration with CI/CD**: Link security test failures to corresponding Jira issues

---

Document Version: 1.0
Last Updated: October 29, 2026
Prepared By: Security Requirements Engineer

Reviewed By: [Pending Review]
Approved By: [Pending Approval]